

SLIDE 1: TITLE SLIDE Good morning, respected judges, faculty members, and fellow participants. I am Anamika Shah, Roll Number 24EACCA009, from the Department of Computer Science and Engineering (AI) at Arya College of Engineering, Kukas, Jaipur. Today, I am excited to present my submission for the Domain Verse 1.0 6-Day Technical Problem-Solving Program under the Agentic AI Track (PS-02). My project is the College AI Helpdesk Agent — an autonomous, evidence-grounded RAG assistant designed for students, faculty, and administrative staff.

SLIDE 2: THE PROBLEM Let's start by addressing the core issue on Slide 2. In any college campus, essential information — like mid-term schedules, department circulars, lab timetables, and RTU syllabus documents — is scattered across different PDFs, web pages, and physical notice boards. Students and staff waste a lot of time every day trying to figure out where to look. The goal of this project is to build a single, reliable point of contact that understands natural language queries, retrieves accurate information instantly, and — most importantly — responds clearly without inventing or hallucinating facts.

SLIDE 3: OBJECTIVE & PS-02 COMPLIANCE Moving to Slide 3, to solve this, I designed a controlled, tool-using helpdesk agent rather than relying on open-ended language generation. The system operates in four steps:

1. Understand: It parses multi-turn queries using LangChain and Google Gemini.
2. Retrieve: It searches across local vector embeddings of RTU syllabus PDFs and structured data.json files.
3. Ground: Every answer is explicitly tagged with its exact source document.
4. Replan: If API rate limits occur or context shifts, it triggers a deterministic local parser.

The system fulfills all core requirements of the PS-02 problem statement, including NLU, role-based access, evidence grounding, and zero-downtime fallbacks.

SLIDE 4: ARCHITECTURE On Slide 4, you can see how a query flows through the system architecture. When a user submits a query through the Streamlit Web UI, the Router & Agent Executor evaluates the request. From there, two retrieval paths converge:

- Primary RAG Path: Performs a dense vector search using ChromaDB and Gemini embeddings.
- Fallback Path: If an API issue occurs, a deterministic Python rule engine parses local JSON and PDF files directly.

Both paths ensure that no answer ever reaches the end user without verified evidence tagging.

SLIDE 5: TECH STACK Slide 5 outlines our technical stack, built entirely using production-grade, free-tier technologies:

- Streamlit powers the multi-portal web UI and role-based authentication.
- LangChain orchestrates agent tools, prompts, and memory.
- Google Gemini provides LLM reasoning and dense vector embeddings.
- ChromaDB serves as our local vector database for fast document retrieval.
- Python 3.10+ and PyPDF handle chunking, indexing, and local fallback parsing.

SLIDE 6: ROLE-BASED ACCESS CONTROL (RBAC) As shown on Slide 6, rather than displaying a generic interface, the system enforces Role-Based Access Control across three tailored portals:

- Student Portal: Focuses on timetables, room lookups, exam alerts, and campus events.
- Faculty Portal: Handles teaching allocations, lab schedules, and semester-wise syllabus breakdown.
- Staff & Admin Portal: Provides operational oversight, circular distribution, and live knowledge base uploading.

SLIDE 7: LIVE DEMO · 01 (ROLE SELECTION & LOGIN) Moving on to Slide 7 for the live demonstration of the application UI. Upon opening the portal, users choose their role: Student, Faculty, or Staff. After selection, a credentialed login screen verifies user identities. Security policies are enforced right at login so that users only access tools and data scoped to their specific role.

SLIDE 8: LIVE DEMO · 02 (STUDENT PORTAL IN ACTION) On Slide 8, we see a live query inside the Student Portal: "When does V semester start?" Notice three key highlights in the agent's output:

1. It extracts exact start dates and mid-term windows.
2. It attaches Verified Evidence Badges linking directly to data.json and rtu 5th sem.pdf.
3. Users can expand the Autonomous Agent Planning Workflow to inspect the step-by-step reasoning behind the response.

SLIDE 9: LIVE DEMO · 03 (FACULTY PORTAL) On Slide 9, in the Faculty Portal, when a teacher asks "What is my timetable?", the agent doesn't just dump plain text. It parses the data and renders a structured Markdown schedule table displaying time slots, subject names, assigned faculty, and room/lab numbers. It maintains the exact same source citation standards across all views.

SLIDE 10: LIVE DEMO · 04 (STAFF & ADMIN PORTAL) Slide 10 demonstrates the Staff & Admin Portal, where queries cover broader operational needs like campus event coordination. Administrators can also upload new PDF circulars or updated JSON schedules on the fly, which automatically re-indexes ChromaDB without requiring a server restart.

SLIDE 11: TRUST & RELIABILITY (ZERO-HALLUCINATION DESIGN) Slide 11 highlights our zero-hallucination execution workflow. Whenever an API bottleneck or connectivity drop occurs, the agent automatically falls back to our deterministic rule-based JSON and PDF parser. This guarantees zero downtime and ensures every generated response is backed by visible proof.

SLIDE 12: CONCLUSION & FUTURE SCOPE Turning to Slide 12, in conclusion, this project delivers a fully compliant, production-ready Helpdesk Agent for PS-02. It combines RAG, multi-portal RBAC, deterministic fallbacks, and transparent planning into a single Streamlit solution.

Looking ahead, the future roadmap includes:

- Automatic push notifications for emergency exam/notice updates.
- Multi-language support for regional languages.
- Voice input integration for interactive campus kiosks.

- Real-time synchronization with official college ERP databases.

SLIDE 13: THANK YOU & Q&A Finally, on Slide 13, thank you for your time and consideration! The source code is hosted on GitHub, and I am now open to any questions or feedback from the panel.